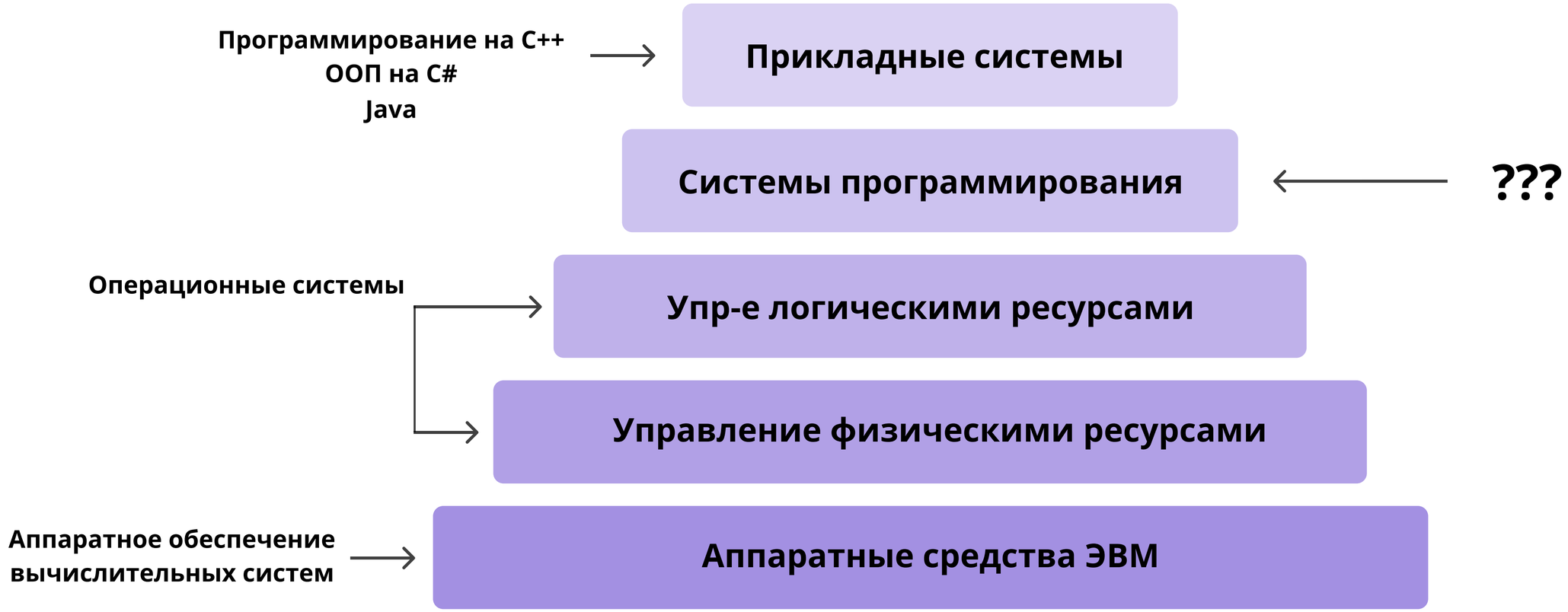
2026

# Платформы и среды выполнения ЯП

NEXIGN x ИТМО

# Место курса в учебном плане




Архитектура вычислительной системы

# Место курса в учебном плане

**Уже знаем**

Ассемблерные инструкции
Регистры, стек вызовов
Процессы, виртуальная память
Системные вызовы
Фон-неймановская модель
Цикл выборки-исполнения
Синтаксис и семантика
C++, C#, Java



**Узнаем в этом курсе**

Как конструкции ЯП транслируются
в машинные инструкции

Как среды выполнения управляют памятью
и исполнением

Как анализировать бинарные файлы
без исходного кода

Как оптимизировать код под своё железо

# Цели и задачи

**Цель**

**Получить глубокое понимание внутренних механизмов современных языков и систем программирования для создания эффективных решений с учётом специфики и ограничений аппаратного обеспечения.**

Для этого нам предстоит:

- Получить представление о жизненном цикле программы от исходного кода до исполнения на процессоре.
- Изучить архитектуру и принципы работы виртуальных машин и сред времени выполнения на примере CLR, JVM и V8.
- Освоить методы анализа объектных файлов, таблиц символов и структур данных времени выполнения.
- Получить практические навыки реверс-инжиниринга исполняемых файлов.
- Изучить особенности языков критических систем и их сред выполнения.
- Освоить методы профилирования и низкоуровневой оптимизации.

# Зачем?

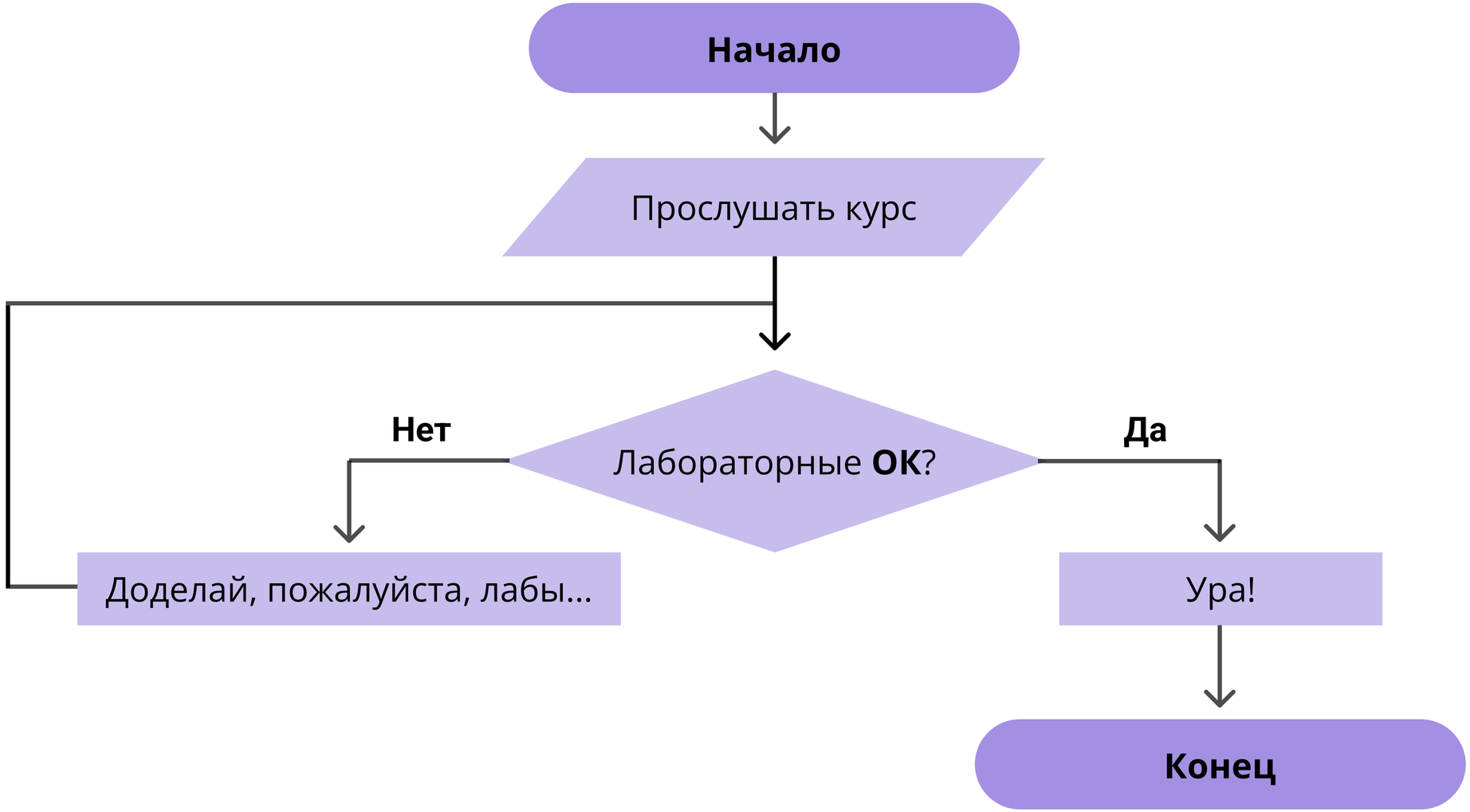
1. Многоязычность производственных систем...
2. Разнообразие и специфика платформ...
3. Экономика облачных вычислений...
4. Безопасность и анализ ПО...
5. Эволюция моделей исполнения...

**Бонус:** общие задачи достаточно хорошо решают ИИ/джуны; специалист, способный решать специфические отраслевые задачи ценится выше.

# Структура курса

| | Раздел |
|---|---|
| 1 | **Жизненный цикл программы:** препроцессинг, компиляция, статическая и динамическая компоновка, объектные файлы. |
| 2 | **Ландшафт памяти:** типизация объектов и размещение в памяти, выравнивание данных и стоимость аллокаций. |
| 3 | **Исполнение программ:** компилируемые, интерпретируемые и JIT-языки, байткод, виртуальные машины |
| 4 | **Ассемблер для разработчика:** чтение, оптимизация, Compiler Explorer. |
| 5 | **Основы реверс-инжиниринга:** анализ исполняемых файлов, дизассемблирование. |
| 6 | **Языки критических систем:** Rust, Ada, Erlang. |
| 7 | **Низкоуровневая оптимизация:** кэш-промахи, векторизация, профилирование. |

# Аттестация


Начало
Прослушать курс
Лабораторные **ОК**?
Нет
Да
Доделай, пожалуйста, лабы...
Ура!
Конец

# Источники

Теория

- Брайант Р., О'Халларон Д. Компьютерные системы: архитектура и программирование. 3-е изд. Главы по компоновке и виртуальной памяти.

C#

- The Book of the Runtime (BOTR). Официальный сборник документации.
- Станислав Сидристый, “Архитектура платформы .NET”. Хардкор на русском. [https://github.com/sidristij/dotnetbook/]

Java

- The Java Virtual Machine Specification, Java SE 26 Edition. Спецификация JVM.

Для самостоятельного изучения

- Материалы конференций: CppCon, RustConf, Highload++.

Раздел 01

# Жизненный цикл программы и модели исполнения



# Определение

**Среда исполнения (runtime)** — слой программного обеспечения между скомпилированной программой и операционной системой или аппаратурой.

Функции среды исполнения:

- Управление памятью (аллокация, сборка мусора).
- Разрешение вызовов и связывание компонентов.
- Обеспечение безопасности и изоляции.
- Оптимизация кода во время исполнения.
- Взаимодействие с операционной системой.

исходный код → промежуточный или машинный код → **среда исполнения** → ОС → железо

# Модели исполнения

Компиляция
в машинный код
(C, C++, Rust)

Промежуточное
представление
+ JIT (C#, Java)

Интерпретация
+ динамическая
компиляция
(JavaScript)

**TL;DR:** в какой момент принимается решение о финальном машинном коде — при сборке, при запуске или во время исполнения?

# Модель 1: компиляция в машинный код

Исходный код → компилятор → машинный код целевой архитектуры → исполняемый файл → загрузка ОС → исполнение.

Характеристики:

- Примеры: C, C++, Rust, Go.
- Финальный машинный код известен до запуска.
- Среда выполнения минимальна: загрузчик ОС, стандартная библиотека, аллокатор.
- Управление памятью: ручное или детерминированное (владение в Rust).

# Модель 2: промежуточное представление

Исходный код → компилятор → промежуточный код (байткод) → среда исполнения (CLR/JVM) → JIT-компилятор → машинный код → исполнение.

**JIT-компиляция (Just-In-Time compilation, Компиляция «точно в нужное время»)** — это производимая во время выполнения процедура преобразования интерпретируемого варианта исполнения программы в программу на языке процессора.

Характеристики:

- Примеры: C#, Java.
- Промежуточный код переносим между архитектурами.
- Финальный машинный код порождается при запуске или во время исполнения.
- Управление памятью автоматическое: сборщик мусора.

# Модель 3: интерпретация

Исходный код → парсер → промежуточное представление → интерпретатор → оптимизирующий компилятор для горячих участков → машинный код → исполнение.

Характеристики:

- Примеры: JavaScript, Python.
- Код анализируется при запуске; нет предварительной сборки.
- Интерпретатор выполняет код сразу; горячие участки компилируются.
- Управление памятью автоматическое: сборщик мусора.

# Сравнение моделей исполнения

| **Критерий** | **Компиляция в машинный код** | **Промежуточный код + JIT** | **Интерпретация + динамическая компиляция** |
|---|---|---|---|
| Примеры языков | C, C++, Rust | C#, Java | JavaScript |
| Момент получения машинного кода | При сборке | При запуске / во время исполнения | Во время исполнения |
| Переносимость | Под целевую архитектуру | Высокая | Высокая |
| Управление памятью | Ручное / детерминированное | Сборщик мусора | Сборщик мусора |
| Среда выполнения | Минимальная | Полноценная VM | Движок |
| Время запуска | Быстрое | Среднее | Зависит от движка |

# Три состояния программы

1. Программа как текст — исходный код, читаемый человеком.
2. Программа как данные — объектный файл, сборка, байткод; структуры, готовые к компоновке или загрузке.
3. Программа как процесс — код, размещённый в памяти и исполняемый процессором.

Вопрос

# Какие преобразования переводят программу из одного состояния в другое?

# Общая схема трансляции

**Конвейер:**
Исходный код → препроцессор → компилятор → ассемблер → объектный файл → компоновщик → исполняемый файл → загрузчик → процесс в памяти.

# Артефакты трансляции

| **Этап** | Команда | Результирующий файл | **Содержимое** |
|---|---|---|---|
| Препроцессинг | gcc -E | .i | Текст с раскрытыми директивами |
| Компиляция | gcc -S | .s | Код на языке ассемблера |
| Ассемблирование | gcc -c | .o | Машинный код, объектный файл |
| Компоновка | gcc | исполняемый файл | Готовая к запуску программа |

# Препроцессинг

**Что делает препроцессор:**

- Раскрытие директив #include...
- Раскрытие макросов #define...
- Обработка условной компиляции (#ifdef, #ifndef, #endif)...
- Удаление комментариев...

```
Исходный код:    #define SQUARE(x) x * x
                 int r = SQUARE(2 + 3);

После раскрытия: int r = 2 + 3 * 3;
```

**Команда:** gcc -E source.c -o source.i

# Компиляция: фронтенд

1. Лексический анализ — разбиение текста на токены: ключевые слова, идентификаторы, операторы, литералы.
2. Синтаксический анализ — построение дерева разбора (абстрактного синтаксического дерева).
3. Семантический анализ — проверка типов, областей видимости, генерация промежуточного представления.

**Команда:** gcc -S source.i -o source.s

# Компиляция: бэкенд

- Оптимизации промежуточного представления.
- Распределение регистров.
- Планирование инструкций.
- Генерация кода на языке ассемблера или машинного кода.

# Ассемблирование

**Что делает ассемблер:**

- Преобразует код на языке ассемблера в машинный код.
- Формирует объектный файл с секциями кода и данных.
- Создаёт таблицу символов.

```
1   Ассемблер:       mov dword [ebp-12], 2
2   Машинный код:    c7 45 f4 02 00 00 00
```

**Команда:** gcc -c source.s -o source.o

# Пример

**Команда:** gcc -Og -o prog main.c sum.c

**Схема действий драйвера:**

1. Препроцессор: main.c → main.i.
2. Компилятор: main.i → main.s.
3. Ассемблер: main.s → main.o.
4. Компоновщик: main.o + sum.o + системные объектные файлы → prog.

# Что дальше?

Объектные файлы → компоновщик → исполняемый файл → загрузчик ОС → процесс в памяти.

# Управляемые (managed) среды

**Список общих черт:**

- Автоматическое управление памятью (сборщик мусора).
- Переносимый промежуточный код (байткод).
- Метаданные и рефлексия.
- Динамическая загрузка компонентов.
- Оптимизация кода во время исполнения (JIT).
- Обработка исключений и безопасность.

**Примеры:** CLR, JVM, V8...

# А какие ещё бывают?..

- Неуправляемые (native / unmanaged): Код компилируется прямо в машинный код и работает с памятью напрямую (как в C или C++).
- Изолированные / контейнеризованные (containers): Окружения уровня ОС (например, Docker), которые изолируют приложение и его зависимости, но используют ядро хоста.
- Облачные / бессерверные (Serverless / FaaS): Среды вроде AWS Lambda или Яндекс.Функций, которые запускают код по событию, полностью скрывая инфраструктуру.

# Байткод

**Тезисы:**

- Байткод — переносимое промежуточное представление, исполняемое виртуальной машиной.
- Позволяет запускать один и тот же код на разных архитектурах без перекомпиляции.
- Используется в JVM, CLR, V8, а также в других системах.
- Виртуальная машина может быть реализована как интерпретатор, как JIT-компилятор или как комбинация обоих подходов.

# Компоновка в управляемых средах

- В компилируемых языках компоновка выполняется на этапе сборки и создаёт исполняемый файл.
- В управляемых средах разрешение зависимостей выполняется динамически, на этапе выполнения: среда находит сборки или классы, загружает их, связывает типы.
- Преимущество: обновление компонентов без пересборки всего приложения.
- Особенность: требуется среда выполнения; возникают накладные расходы на загрузку.

# TL;DR

1. Программа проходит цепочку преобразований: исходный код, промежуточное представление, машинный код, исполнение.
2. Среда выполнения — посредник между высокоуровневыми абстракциями языка и аппаратурой.
3. Существуют различные модели исполнения: компиляция в машинный код, промежуточный код с JIT, интерпретация с динамической компиляцией.
4. Границы между моделями размываются в современных реализациях.
5. Понимание этих механизмов необходимо для оптимизации, диагностики ошибок и обеспечения безопасности.

# Проверить себя

1. Чем среда выполнения отличается от компилятора?
2. В какой момент получается финальный машинный код в компилируемых языках, в языках с JIT, в интерпретируемых языках?
3. Что такое байткод и какие преимущества он даёт?
4. Чем объектный файл отличается от исполняемого?
5. Какие этапы проходит исходный код до получения исполняемого файла?